

Integral Literal contd....

An integral literal we can represent in four different forms:
a) Decimal Literal (Base is 10)
b) Octal Literal (Base is 8)
c) Hexadecimal Literal (Base is 16 [0-9 and a - f OR A - F])
d) Binary Literal (Base 2) [Available from JDK 1.7v]

Decimal Literal:

By default, our numeric literals are decimal literal. Here base is 10 so, It accepts 10 digits i.e. from 0-9.

Example:

```
int x = 20;  
int y = 123;  
int z = 234;
```

Octal Literal:

If any Integral literal starts with 0 (Zero) then it will become octal literal. Here base is 8 so it will accept 8 digits i.e. 0 to 7.

Example:

```
int a = 018; //Invalid because it contains digit 8 which is out of range  
int b = 0777; //Valid  
int c = 0123; //Valid
```

Hexadecimal Literal:

If any integral literal starts with 0X or 0x (Zero capital X Or 0 small x) then it is hexadecimal literal. Here base is 16 so it will accept 16 digits i.e. 0 to 9 and A to F OR [a to f]

Example:

```
int x = 0X12; //Valid  
int y = 0xadd; //Valid  
int z = 0XFACE; //valid  
int a = 0Xage; //Invalid because character 'g' out of range
```

Binary Literal:

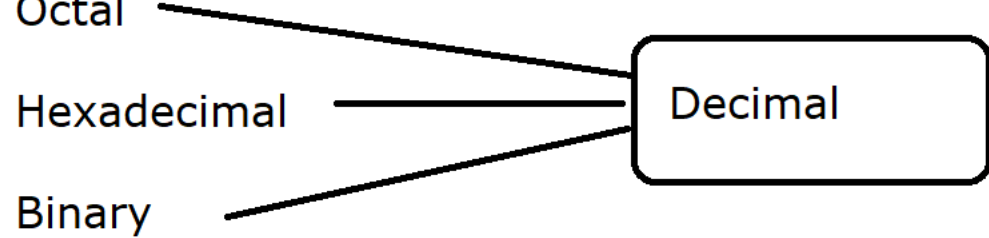
If an Integral literal starts with 0B (Zero capital B) or 0b (Zero small b) then it will become Binary literal. Binary literal is available from JDK 1.7v.

Here base is 2 so it will accept 2 digits i.e. 0 and 1 only.

Example:

```
int x = 0B101; //valid  
int y = 0b111; //Valid  
int z = 0B112; //Invalid [2 is out of range]
```

The default type is **decimal literal** so to generate the output for any different literal JVM converts into decimal literal.



//Programs :

$$\begin{array}{c} 2 \quad 1 \quad 0 \\ \diagdown \quad | \quad / \\ 5 \quad 2 \quad 3 \end{array}$$
$$500 + 20 + 3$$
$$5 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$$

Octal Literal program :

015 -> Here '0' describes that it is an octal literal.

$$\begin{array}{c} 1 \quad 0 \\ \diagdown \quad / \\ (1 \quad 5) \\ 8 \end{array} = (?)_{10}$$

$$1 \times 8^1 + 5 \times 8^0$$
$$8 + 5 = 13$$

Octal.java

```
void main()  
{  
    int x = 015;  
    IO.println(x);  
}
```

Working with hexadecimal :

0Xadd -> Here 0X describes that It is hexadecimal literal.

$$\begin{array}{c} 1 \\ | \\ 2 \quad (a \quad d \quad d) \quad 0 \\ \diagdown \quad | \quad / \\ 16 \end{array}$$

0 - 9
A = 10
B = 11
C = 12
D = 13
E = 14
F = 15

$$a \times 16^2 + d \times 16^1 + d \times 16^0$$
$$10 \times 256 + 13 \times 16 + 13 \times 1$$
$$2560 + 208 + 13$$
$$2781$$

Hexa.java

```
void main()  
{  
    int x = 0Xadd;  
    IO.println(x);  
  
    int y = 0XFACE;  
    IO.println(y);  
}
```

Working with Binary Literal :

0B101 -> 0B describes that It is binary literal

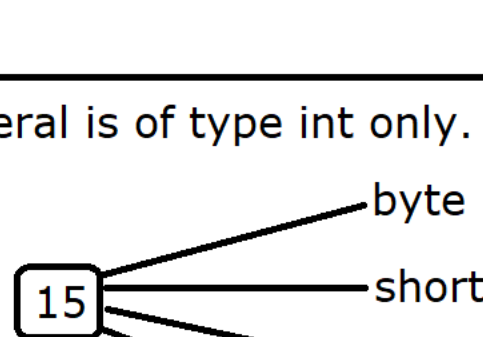
$$\begin{array}{c} (1 \quad 0 \quad 1) \\ 2 \end{array} = (?)_{10}$$

$$1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$
$$4 + 0 + 1$$
$$5$$

Binary.java

```
void main()  
{  
    int x = 0B101;  
    IO.println(x);  
}
```

* By default, Every integral literal is of type int only.



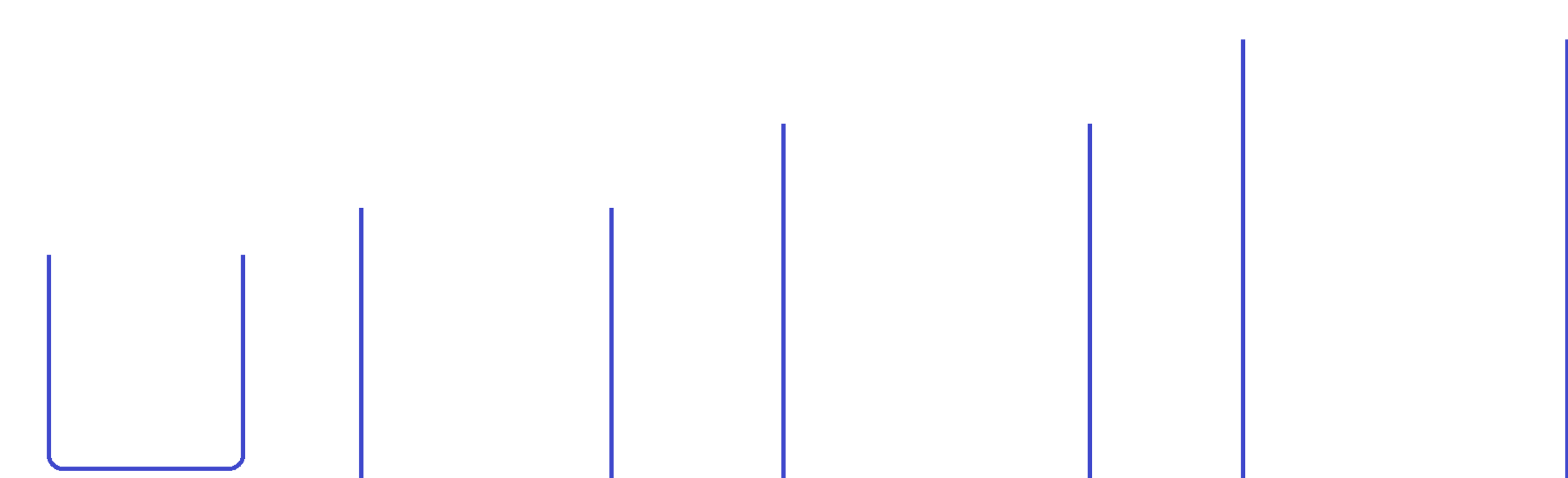
Integral Literal Memory Representation :

byte b = 5; 00000101 (8 bits)

short s = 5; 00000000 00000101 (16 bits)

int x = 5; 00000000 00000000 00000000 00000101 (32 bits)

long y = 5; 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000101 (64 bits)



byte & short data type range :

byte : -128 to 127	$[-2^7 \text{ To } 2^7 - 1]$
short : -32768 to 32767	$[-2^{15} \text{ To } 2^{15} - 1]$

Different cases :

Case 1 :

byte b = 90; //[Javac will automatically convert the value 90 into byte type because the value is within the range]

Case 2 :

byte b = 130; //[Compilation Error because value is out the range, range is : **-128 to 127**]

Case 3:

short s = 32767; //[Javac will automatically convert 32767 into short type]

Case 4:

long s = 9812345678; //[CE, Integer number too large (Out of the range)]

Case 5 :

long mob = 9812345678L; //9812345678 is of type long (64 bits)

Case 6 :

```
void main()  
{  
    m1(1); //error 1 is of type int only  
}
```

```
void m1(byte b)  
{  
    IO.println("byte");  
}
```

By default, every integral literal is of type **int** only. **byte and short are below than int** so we can assign integral literal (Which is by default int type) to byte and short but the values must be within the range. [for byte -128 to 127 and for short -32768 to 32767]

Actually, whenever we are assigning integral literal to byte and short data type then compiler internally converts into corresponding type.

byte b = (byte) 12; [Compiler is converting int to byte]
short s = (short) 12; [Compiler is converting int to short]

In order to represent long value explicitly we should use either L OR l (Capital L OR Small l) as a suffix to integral literal.

According to IT industry, we should use **L** because l (small l) looks like digit 1.

Test4.java

```
/* By default every integral literal is of type int only*/  
  
void main()  
{  
    byte b = 128;  
    IO.println(b);  
  
    short s = 32768;  
    IO.println(s);  
}
```